Trabalho Prático Linguagens de Programação

Participantes:

- Leandro Alfredo Carlos
- Lucas Pereira de Lira

Especificações da Base de Dados:

- Nome:
  - Spotify Dataset 1922-2021, ~600k Tracks
- Estrutura:
  - tracks.csv
    - Primário :
      - id (Id da trilha gerada pelo Spotify)
    - Numérico :
      - acousticness (varia de 0 a 1)
      - danceability (varia de 0 a 1)
      - energy (varia de 0 a 1)
      - duration_ms (número inteiro normalmente varia de 200k a 300k)
      - instrumentalness (varia de 0 a 1)
      - valence (varia de 0 a 1)
      - popularity (varia de 0 a 100)
      - tempo (flutuação normalmente variando de 50 a 150)
      - liveness (varia de 0 a 1)
      - loudness (flutuação tipicamente variando de -60 a 0)
      - speechiness (varia de 0 a 1)
    - Fictício:
      - mode (0 = Menor, 1 = Maior)
      - explicit (0 = sem conteúdo explícito, 1 = conteúdo explícito)
    - Categórico :
      - key (todas as chaves na oitava codificadas como valores que variam de 0 a 11, começando em C como 0, C # como 1 e assim por diante ...)
      - timesignature (a assinatura de tempo prevista, mais tipicamente 4)
      - artists (lista dos artistas mencionados)
      - artists_id (Ids dos artistas mencionados)
      - release_date (data de lançamento principalmente no formato aaaa-mm-dd, no entanto, a precisão da data pode variar)
      - name (nome da música)
  - artists.csv
    - id (id do artista)
    - name (nome do artista)
    - followers (número total de seguidores do artista)
    - popularity (popularidade de determinado artista com base em todas as suas faixas)
    - genres (gêneros associados a este artista)
  - dict_artists.json {

```
            "qualquer": [
                "primeiro",
                "segundo",
                "terceiro",
                ...,
                "enésimo"
            ],
            "em branco": [],
            "primeiro: [
                " qualquer ",
                " terceiro ",
                " segundo
            ],
            ...

    }
```

  - (As listas são classificadas em ordem decrescente, então "primeiro" é o mais semelhante a "qualquer" e "segundo" o segundo é mais e assim por diante... O número de artistas pode variar de 0 a 20)
- Link de Acesso: [https://www.kaggle.com/yamaerenay/spotify-dataset-19212020-160k-tracks?select=dict_artists.json (https://www.kaggle.com/yamaerenay/spotify-dataset-19212020-160k-tracks?select=dict_artists.json)](https://www.kaggle.com/yamaerenay/spotify-dataset-19212020-160k-tracks?select=dict_artists.json)

Considerações Gerais

- Para este trabalho decidimos eliminar os valores nulos encontrados, outras técnicas foram desconsideradas.
- A coluna "artists" possui vetores em seus registros, com isto, foi necessário desmembrar esses valores para que fosse possível demonstrar resultados mais fiéis. Porém a população é muito grande, portanto fazer uma enumeração completa de todos os valores na população tornou-se inviável e muito demorado. Com isto, utilizamos uma amostra (subconjunto) dos dados (observe situações do tipo .head(X), onde "x" é a amostra), de tamanho manejável e possível de se demonstrar uma análise fidedigna.
- Nas células onde há códigos, alguns comentários foram adicionados, favor considere-os.
- Dependendo do tamanho do dataframe é necessário inciar o notebook com o seguinte comando:
  - jupyter notebook --NotebookApp.iopub_data_rate_limit=1.0e10

In [1]:

```
import pandas  as pd
import seaborn as sbn
import matplotlib.pyplot as plt

%matplotlib inline

import numpy as np

# Observação: Para utilizar o plotly no Jupyter é necessário instalá-lo, sendo assim, abra
# seguinte comando:
# conda install -c https://conda.anaconda.org/plotly plotly
# ou com o comando pip install plotly

import plotly.express as px
```

In [2]:

```
artist_popularity_metrics = pd.read_csv('Spotify Dataset 2021/artists.csv')
audio_features_of_tracks  = pd.read_csv('Spotify Dataset 2021/tracks.csv')
related_artists           = pd.read_json('Spotify Dataset 2021/dict_artists 1.0.json')
df_data_by_year_o         = pd.read_csv('Spotify Dataset 2021/data_by_year_o.csv')
```

In [3]:

```
artist_popularity_metrics.columns = ['id_artist', 'followers', 'genres', 'name', 'popularit
audio_features_of_tracks.columns = ['id', 'name', 'popularity', 'duration_ms', 'explicit',
```

Lista de Questões

- Inicialmente foi realizado o tratamento dos dados, onde verificamos se existem valores nulos nas colunas do dataframe. Para isso, foi utilizado o comando DataFrame.isna() que detecta os valores nulos no conjunto de dados, logo após, foi aplicado a função .sum() que realiza a contagem desses valores, como mostrado na célula a seguir.

In [4]:

```
print(audio_features_of_tracks.isna().sum())
```

```
id                  0
name               71
popularity          0
duration_ms         0
explicit            0
artists             0
id_artist           0
release_date        0
danceability        0
energy              0
key                 0
loudness            0
mode                0
speechiness         0
acousticness        0
instrumentalness    0
liveness            0
valence             0
tempo               0
time_signature      0
dtype: int64
```

- Nota-se na saída do código acima que a junção dos comandos nos fornece a quantidade de valores nulos separados por coluna, o que facilitou a sua correção. Conforme destacado no início do trabalho, optamos por eliminar todos os registros que possuem valores nulos. Observa-se que apenas a coluna "name" possui valores nulos, os quais serão eliminados com o comando abaixo.
- Antes de prosseguir será verificado quantos registros existem no dataframe, para isso, utilizamos o comando DataFrame.shape que nos apresentará informações sobre a dimensionalidade do nosso dataframe.

In [5]:

```
audio_features_of_tracks.shape
```

Out[5]:

```
(586672, 20)
```

- O dataframe possui 586.672 registros.
- Aplicamos o código para eliminação dos registros nulos encontrados e os atribuimos a um novo dataframe, denominado "not_null".

In [6]:

```
not_null = audio_features_of_tracks[ audio_features_of_tracks['name'].isnull() == False ]
```

- Feito isto, executamos o comando verificar se os comandos nulos realmente foram eliminados.

In [7]:

```
not_null.shape
```

Out[7]:

```
(586601, 20)
```

- Nota-se a redução de 71 registros.
- Após os tratamentos desses dados, definiremos e analisaremos as questões a serem respondidas.

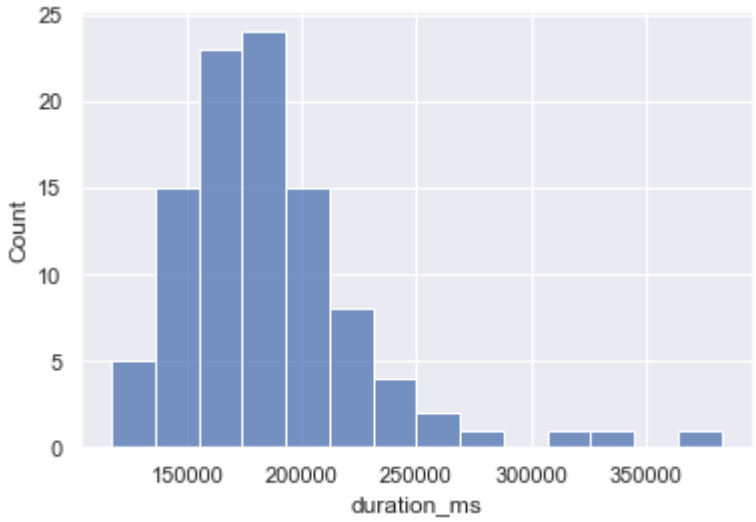
1 - Qual a tendência na duração das 100 músicas mais populares no ano de 2021?

In [8]:

```
# Filtrando as músicas com lançamento em 2021
_2021_songs = not_null[ not_null['release_date'].str.replace(' ', '').str.contains('2021')

# Listando as 100 músicas mais populares de 2021
_100_most_popular_songs = _2021_songs.sort_values(by='popularity', ascending=False).head(10

df = _100_most_popular_songs['duration_ms']

# Gerando o histograma da distribuição da duração das músicas
sbn.set_theme(style="darkgrid")
sbn.histplot(df)
```

Out[8]:

```
<AxesSubplot:xlabel='duration_ms', ylabel='Count'>
```

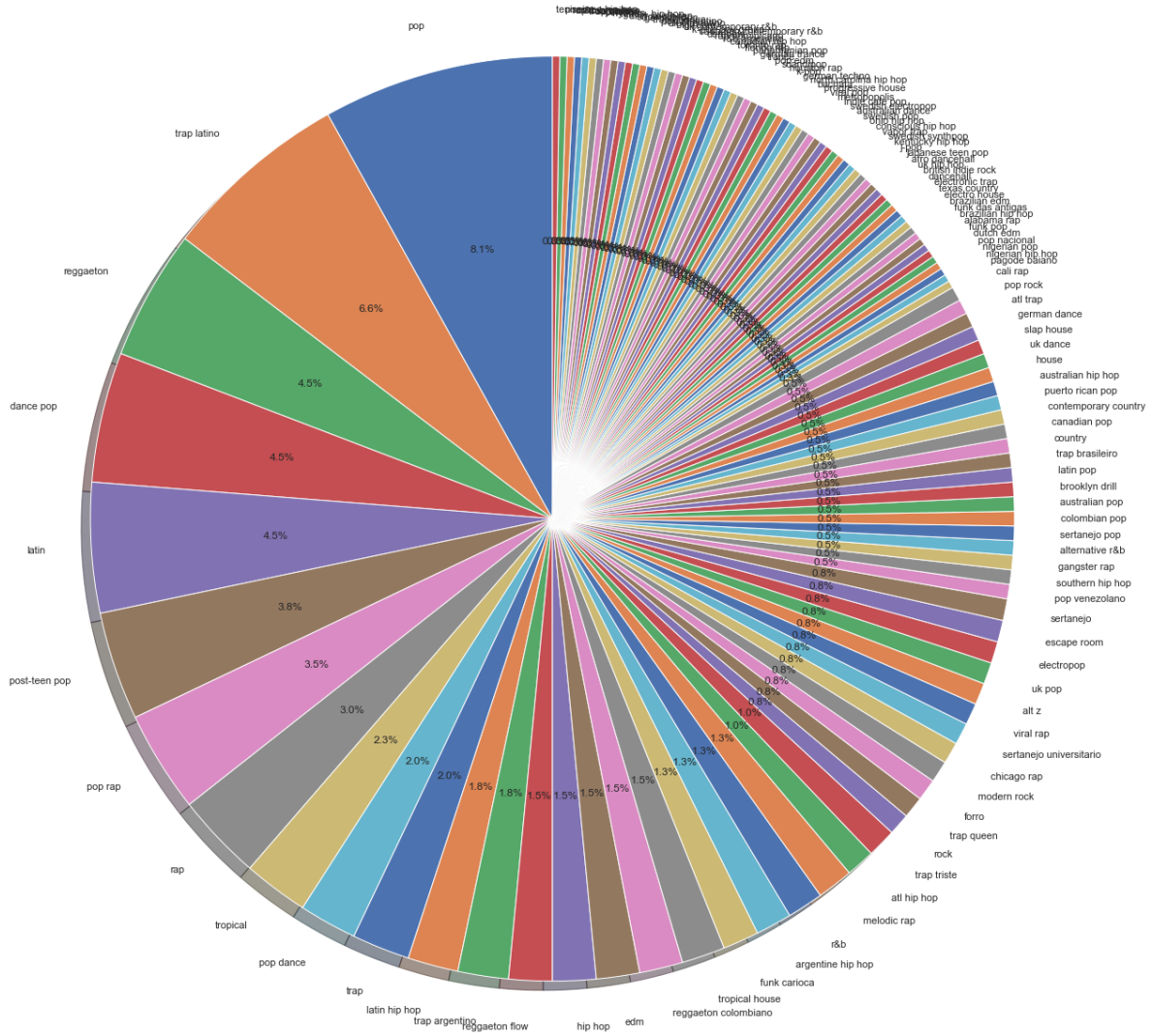


- Um histograma é uma ferramenta de visualização de dados clássica, muito utilizada em divesas áreas, sua estrutura representa a distribuição de uma ou mais variáveis através da contagem do número de ocorrências de um determinado intervalo. Além disso, podemos utilizar o processamento de imagens como exemplo para ilustrar diversas outras formas de utilização, como na equalização de histograma ou histogram stretching para manipulação do contraste da imagem, além de diversas outras.
- Análise: Observou-se através do histograma acima que as músicas de maior popularidade têm, em sua maioria, uma duração que varia entre 150000 ms (2,5 minutos) e 200000 ms (3,33 minutos). Além disso pode-se dizer que o gráfico não segue uma distribuição normal, sua distribuição está enviesada, ou seja, tende a valores mais a esquerda.

2 - Quais os gêneros de música predominantes entre os artistas que tem participação nas 100 música mais populares de 2021?

In [9]:

```
# Filtrando as músicas com lançamento em 2021
# As datas podem conter somente o ano, além de outros formatos, sendo assim utilizo um cont
_2021_songs = not_null[ not_null['release_date'].str.replace(' ', '').str.contains('2021')

# Listando as 100 músicas mais populares de 2021
_100_most_popular_songs = _2021_songs.sort_values(by='popularity', ascending=False).head(10

# Observação: Devido a coluna de artistas possuir como valores objetos do tipo vetor, torno
# processar os dados da seguinte forma

list_id_artists = []

# Obtendo o id dos artistas que participam nas 100 músicas mais populares
for row in _100_most_popular_songs['id_artist']:

    aux_list = row.replace('[', '').replace(']', '').replace("'", '').replace(' ', '').spli

    # Verificando para não incluir duplicados
    for i in range(0, len(aux_list)):
        it_found = False
        j = 0
        while j < len(list_id_artists) and it_found == False:
            if aux_list[i] == list_id_artists[j]:
                it_found = True
            j+=1
        if it_found == False:
            list_id_artists.append(aux_list[i])

genre_list = []

# Obtendo os gêneros de música dos dos artistas (Como são para artistas únicos estes podem
for i in range(0, len(list_id_artists)):
    for row in artist_popularity_metrics[artist_popularity_metrics['id_artist'].replace(' '
        aux_list = row.replace('[', '').replace(']', '').replace("'", '').split(',')

        for j  in range(0, len(aux_list)):
            if aux_list[j].replace(' ', '') != '':
                genre_list.append(aux_list[j].strip())

df_genre = pd.DataFrame(genre_list)

df_genre2 = df_genre.value_counts().sort_values(ascending=False)
labels = pd.unique(pd.Series(genre_list))

aux_list_dataframe = []

# Definindo a lista resultante
for i in range(0, len(labels)):
    aux_list_dataframe.append([labels[i], df_genre2[labels[i]]])

df_result = pd.DataFrame(aux_list_dataframe, columns=['name', 'value'])
df_result = df_result.sort_values(by='value', ascending=False)

plt.rcParams['figure.figsize'] = [20, 20]
fig1, ax1 = plt.subplots()
ax1.pie(df_result['value'], labels=df_result['name'], autopct='%1.1f%%', shadow=True, start
ax1.axis('equal')
plt.show()
```



- O gráfico de pizza, também conhecido como gráfico de torta ou gráfico de setores é um tipo de gráfico onde os valores de cada categoria estatística representados são proporcionais as suas respectivas frequências.
- Análise: Observa-se no gráfico acima a proporção de cada genêro de música que estão vinculadas aos cantores que tiveram participação nas 100 músicas mais populares de 2021. Conforme podemos visualizar os três primeiros gêneros mais frequêntes entre os ouvintes são pop com 8.1%, trap latino com 6.6% e reggaeton com 4.5%.

- Até este ponto vimos que o conjunto de dados contém mais de 586.000 músicas, coletadas a partir da API do Spotify. Os recursos incluem, música, artista, data de lançamento, bem como algumas características da música, como, acústico, dançante, volume, andamento e assim por diante. Afim de reduzir a carga de processamento dos códigos que sucedem este comentário iremos eliminar algumas colunas do nosso dataframe e podemos fazê-lo através da chamada do método drop, como mostrado na célula a seguir.

In [10]:

```
new_df = not_null.copy()
new_df.drop([ 'id', 'explicit', 'key', 'mode'], axis = 1, inplace = True)
new_df.head()
```

Out[10]:

	name	popularity	duration_ms	artists	id_artist	release_date
0	Carve	6	126903	['Uli']	['45tIt06XoI0Iio4LBEVpls']	1922-02-22
1	Capítulo 2.16 - Banquero Anarquista	0	98200	['Fernando Pessoa']	['14jtPCOoNZwquk5wd9DxrY']	1922-06-01
2	Vivo para Quererte - Remasterizado	0	181640	['Ignacio Corsini']	['5LiOoJbxVSAMkBS2fUm3X2']	1922-03-21
3	El Prisionero - Remasterizado	0	176907	['Ignacio Corsini']	['5LiOoJbxVSAMkBS2fUm3X2']	1922-03-21
4	Lady of the Evening	0	163080	['Dick Haymes']	['3BiJGZsyX9sJchTqcSA7Su']	1922

3 - Analise se há correlação (similaridade) entre a coluna "danceability" e a coluna "valence"?

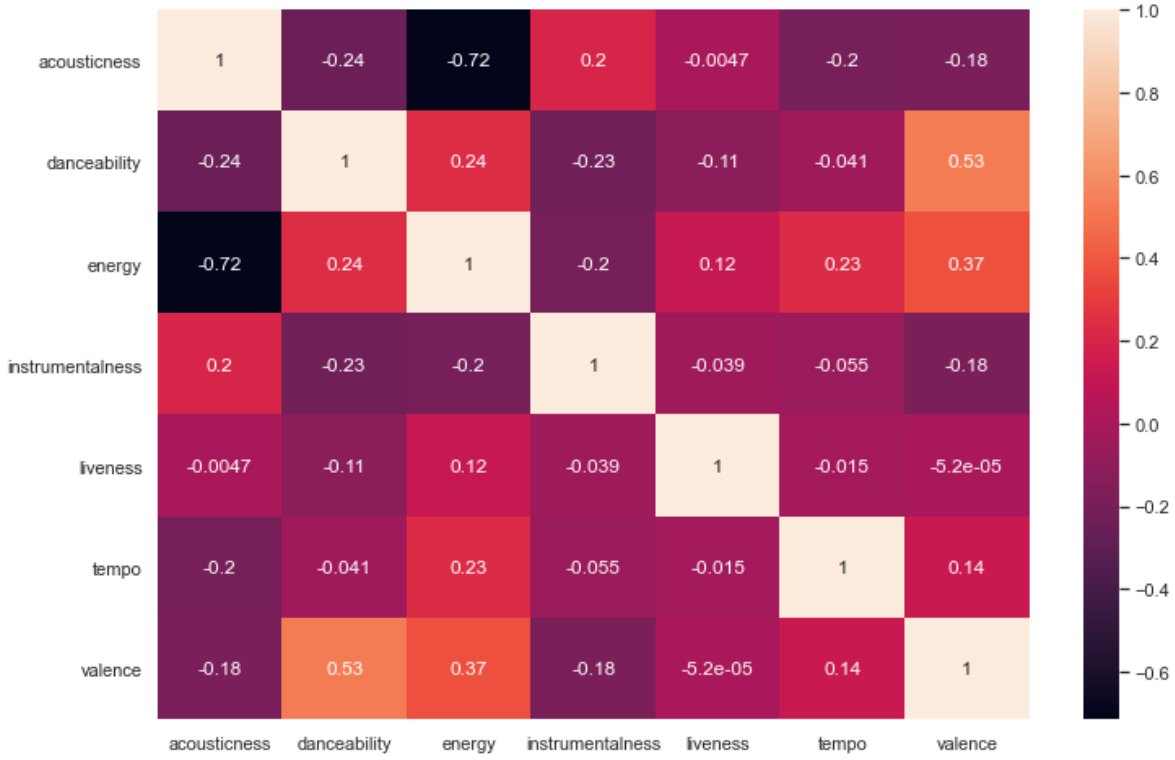
- O conjunto de dados inclui diversas métricas. Algumas delas deixam evidente seu significado, como ritmo, volume, energia, acústica, vivacidade, fala e podem ser relevantes na análise de dados. À primeira vista, as colunas "danceability" e "valence" parecem estar relacionadas, pois a "valence" é uma medida que descreve a positividade musical transmitida ao ouvinte, sendo assim, podemos deduzir que a positividade pode influenciar na decisão dos ouvintes quanto a dançar ou não uma determinada música. A fim de analisar a relação entre as colunas do dataframe, utilizaremos o método DataFrame.corr(), este nos fornece um conjunto de medidas que definem o nível de correlação entre as colunas. O código apresentado a seguir explora essa funcionalidade e apresenta os resultados de uma forma mais adequada.

In [11]:

```
corr = new_df[['acousticness', 'danceability', 'energy', 'instrumentalness', 'liveness', 't
sbn.set(style='darkgrid')
plt.figure(figsize=(12,8))
sbn.heatmap(corr, annot=True)
```

Out[11]:

```
<AxesSubplot:>
```



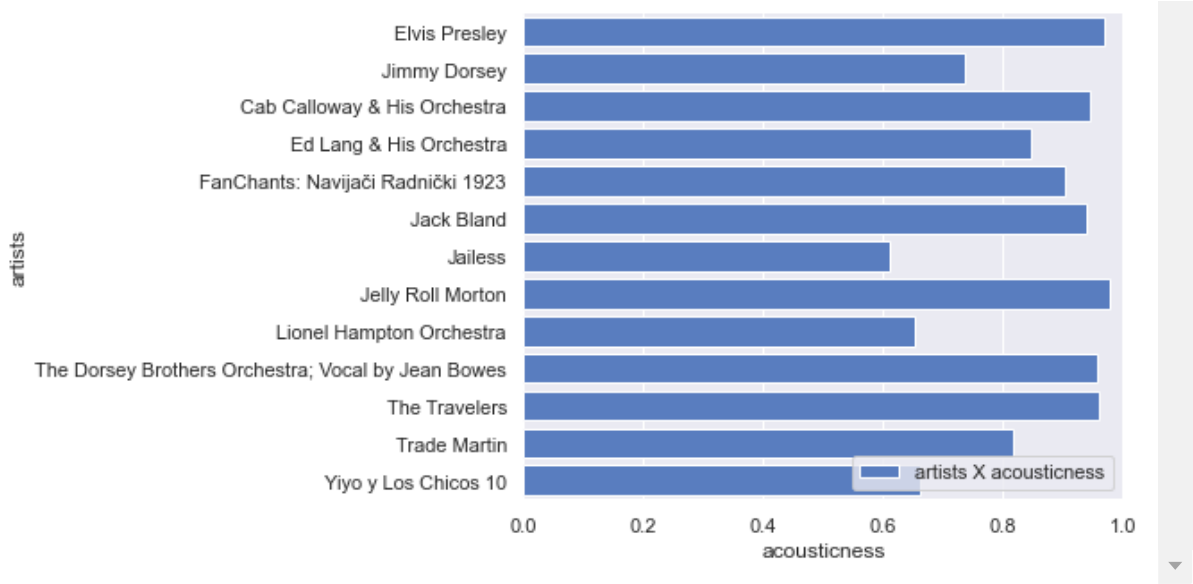
- Análise
  - Usamos o Heatmap (ou mapa de calor), que utiliza as cores como referência para facilitar o entendimento das informações, para representar graficamente o resultado do dateFrame acima. Neste gráfico, temos uma barra na lateral direita contendo valores positivos (cores mais "claras ou quentes") e negativos (cores mais "escuras ou frias"). Após a demonstração dos dados graficamente, pode-se confirmar a correlação entre as colunas "danceabiliaty" e "valence" com o valor positivo 0.57.

E também é visível uma correlação nos estilos musicais "danceability" e "energy" ou "acousticness" e "instrumentalness", pois são valores positivos e próximos do valor 0.5. Ao contrário temos, "energy" e "acousticness", que mostram-se estilos musicais muito diferentes, pois, se relacionam negativamente em -0.77.

4 - Apresente os artistas que produzem músicas tanto enérgicas (energy) quanto acústicas (acousticness), levando em conta que um artista nesta categoria deva ter um valor médio para ambas as colunas de 0.5 ou superior.

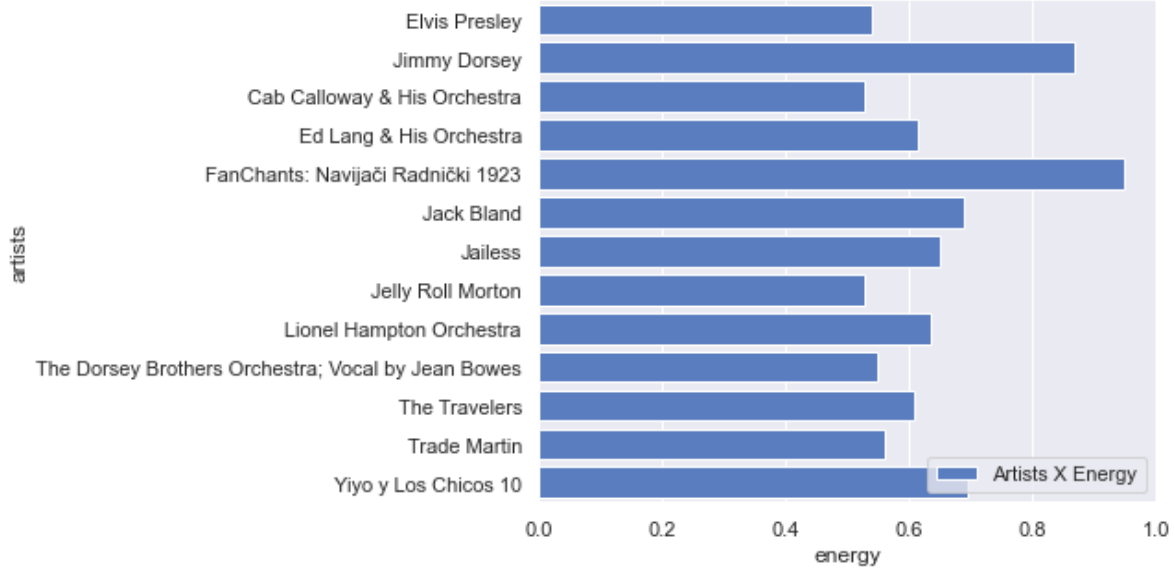
In [15]:

```
list_data = []

aux_df = new_df[['artists','energy','acousticness']]

#Observação:
# A coluna "artists" possui vetores em seus registros, com isto, foi necessário desmembrar
# que fosse possível demonstrar resultados mais fiéis.
# Porém a população é muito grande, portanto fazer uma enumeração completa de todos os valo
# tornou-se inviável e muito demorado.
# Portanto utilizamos uma amostra (subconjunto) dos dados, de tamanho manejável e possível
# uma análise fidedigna.

aux_df = aux_df.head(3000)

for i in range(len(aux_df)):
    list_artists = aux_df.values[i][0].replace('[', '').replace(']', '').replace("'", '').s
    for artist in list_artists:
        list_data.append([artist, aux_df.values[i][1], aux_df.values[i][2]])

df_result = pd.DataFrame(list_data, columns=['artists', 'energy', 'acousticness'])

aux_df = df_result.groupby('artists').mean()

aux_df = aux_df[aux_df['energy'] > 0.5]
aux_df = aux_df[aux_df['acousticness'] > 0.5]

new_df_list = []

for artist in aux_df.index.tolist():
    new_df_list.append([artist])

for i in range(len(aux_df)):
    new_df_list[i].append(str(aux_df.values[i][0]))
    new_df_list[i].append(str(aux_df.values[i][1]))

new_df_list = pd.DataFrame(new_df_list, columns=['artists', 'energy', 'acousticness'])

new_df_list['artists'] = new_df_list['artists'].astype("string")
new_df_list['energy'] = new_df_list['energy'].astype(float)
new_df_list['acousticness'] = new_df_list['acousticness'].astype(float)

f, ax = plt.subplots(figsize=(6, 5))

sbn.set_color_codes("muted")
sbn.barplot(x="acousticness", y="artists", data=new_df_list,
            label="artists X acousticness", color="b")

ax.legend(ncol=2, loc="lower right", frameon=True)
ax.set(xlim=(0, 1), ylabel="artists",
       xlabel="acousticness")
sbn.despine(left=True, bottom=True)
```



In [16]:

```
f, ax = plt.subplots(figsize=(6, 5))
sbn.set_color_codes("muted")
sbn.barplot(x="energy", y="artists", data=new_df_list,
            label="Artists X Energy", color="b")

ax.legend(ncol=2, loc="lower right", frameon=True)
ax.set(xlim=(0, 1), ylabel="artists",
       xlabel="energy")
sbn.despine(left=True, bottom=True)
```



- Análise
  - Podemos entender que Músicas do tipo Energy são totalmente contrárias as músicas do tipo Acousticness. De um lado músicas agitadas, com muito ritmo, por exemplo, e do outro lado músicas mais calmas e lentas. Geralmente esta versatilidade no ritmo é difícil de se encontrar no mesmo artista. E conforme os dados exibidos acima vemos que realmente os artistas que mais produzem músicas do tipo energy são os que menos produzem ou cantam músicas do tipo acousticness. Porém, realizamos um filtro onde destacamos os artistas que possuem esta versatilidade e talento ao produzir, lançar ou cantar músicas com estilos ritmicos tão distintos.

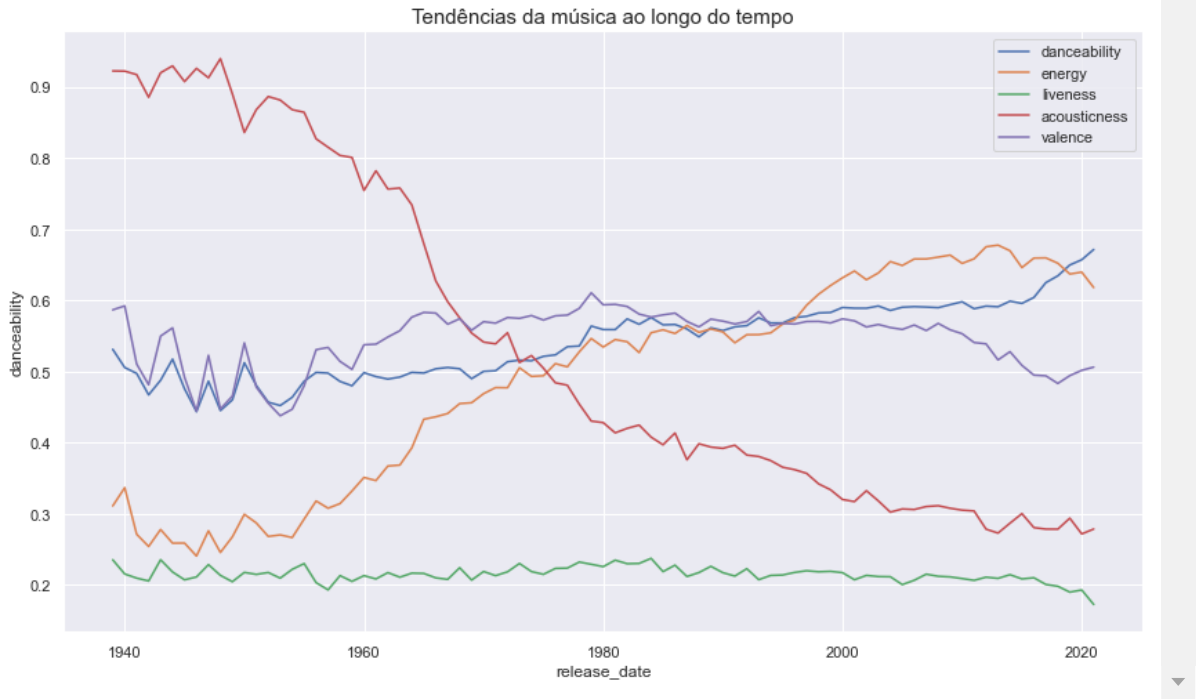
5 - Visão geral das tendências musicais entre os anos de 1940 a 2021.

- A coluna release_date está formata com a data completa, ex.: 1900-01-01. Porém, a análise será realizada entre a média das características musicais por ano. Com isto, iremos formatar a data de tal forma a manter somente o ano (um valor inteiro).

In [17]:

```
df_6 = new_df.copy()

#Observação:
# A coluna "artists" possui vetores em seus registros, com isto, foi necessário desmembrar
# que fosse possível demonstrar resultados mais fiéis.
# Porém a população é muito grande, portanto fazer uma enumeração completa de todos os valo
# tornou-se inviável e muito demorado.
# Portanto utilizamos uma amostra (subconjunto) dos dados, de tamanho manejável e possível
# uma análise fidedigna.

df_6['release_date'] = pd.to_datetime(df_6['release_date'], errors='coerce')
df_6['release_date'] = df_6['release_date'].dt.strftime('%Y')

df_6['release_date'] = df_6['release_date'].astype(int)

df_6 = df_6[df_6['release_date'] >= 1939]

year_avg = df_6[['danceability','energy','liveness','acousticness', 'valence','release_date

print(year_avg.head())

# Pronto, para as cinco medidas diferentes, obtivemos os valores médios anuais.
# Agora, primeiro vou criar um figure e adicionar uma linha para cada tendência/caracterist
plt.figure(figsize=(14,8))
plt.title("Tendências da música ao longo do tempo", fontsize=15)
lines = ['danceability','energy','liveness','acousticness','valence']
for line in lines:
    ax = sbn.lineplot(x='release_date', y=line, data=year_avg)
plt.legend(lines)

plt.show()
```

```
              danceability    energy  liveness  acousticness   valence
release_date
1939              0.531176  0.311143  0.235362      0.922498  0.586671
1940              0.505680  0.336756  0.215495      0.922189  0.592268
1941              0.497355  0.271278  0.209660      0.917301  0.510550
1942              0.467199  0.254129  0.205647      0.885475  0.481395
1943              0.487777  0.278011  0.235460      0.919878  0.549790
```



- Análise
  - Com os dados apresentados acima, vemos que o tipo acousticness por volta dos anos 40 foi tendência, porém a partir de 1960 teve um queda acentuada que perdura até os dias de hoje. Ao contrário, vemos, por exemplo, as músicas do tipo energy que no começo dos anos 40 até meados dos anos 60 não era tendência, mas a partir dos anos 60 começou a ganhar popularidade e virou tendência o que perdura até os dias de hoje. O que também é interessante observar que as tendências musicas ditam "moda", pois quanto o maior consumo dos ouvintes naquela caracterísitca nota-se uma maior produção da mesma. Nisto, entendemos que realmente há uma preocupação das gravadoras/produtores em seguir o mercado e lançar artista quem seguem o que já está "nichado".

6 - Artistas, vamos ver os 10 principais artistas com mais músicas no conjunto de dados.

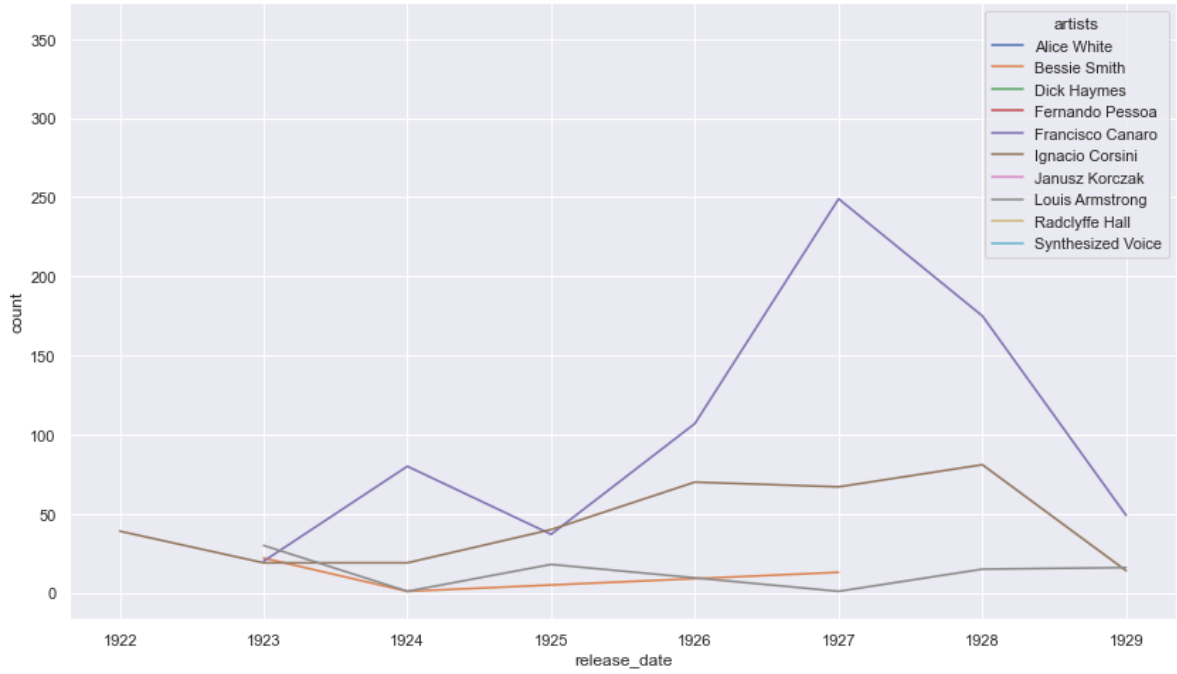
Alguns deles produzem muitas canções, enquanto há alguns artistas com pouquíssimas. a) Vamos ver os 10 principais artistas com mais músicas no conjunto de dados.

In [18]:

```
df_7 = new_df.copy()

#Observação:
# A coluna "artists" possui vetores em seus registros, com isto, foi necessário desmembrar
# que fosse possível demonstrar resultados mais fiéis.
# Porém a população é muito grande, portanto fazer uma enumeração completa de todos os valo
# tornou-se inviável e muito demorado.
# Portanto utilizamos uma amostra (subconjunto) dos dados, de tamanho manejável e possível
# uma análise fidedigna.

df_7 = df_7.head(5000)

list_data = []

for i in range(len(df_7)):

    list_artists = df_7.values[i][3].replace('[', '').replace(']', '').replace("'", '').spl

    for artist in list_artists:

        name = artist.replace('"', '').replace("'", '').strip()

        if (name == 'Janusz Korczak' or name == 'Radclyffe Hall' or name == 'Alice White' o
            or name == 'Francisco Canaro' or name == 'Fernando Pessoa' or name == 'Ignacio
               or name == 'Louis Armstrong' or name == 'Bessie Smith' or name == 'Dick Haym
            list_data.append([name, int(df_7.values[i][5].split('-')[0]), name])

df_7 = pd.DataFrame(list_data, columns=['artists', 'release_date', 'aux_artists'])

df_7['artists'] = df_7['artists'].astype('string')
df_7['aux_artists'] = df_7['aux_artists'].astype('string')

df_7 = df_7.groupby(['artists', 'release_date']).count()

new_df_list = []

for values in df_7.index.tolist():
    new_df_list.append([values[0], values[1]])

for i in range(len(df_7)):
    new_df_list[i].append(str(df_7.values[i][0]))

new_df_list = pd.DataFrame(new_df_list, columns=['artists', 'release_date', 'count'])

new_df_list['artists'] = new_df_list['artists'].astype('string')
new_df_list['count'] = new_df_list['count'].astype(int)

# new_df_list.sort_values(by='count', ascending=False).head(10)

plt.figure(figsize=(14, 8))
sbn.lineplot(x='release_date', y='count', hue='artists', data=new_df_list)
```

Out[18]:

```
<AxesSubplot:xlabel='release_date', ylabel='count'>
```



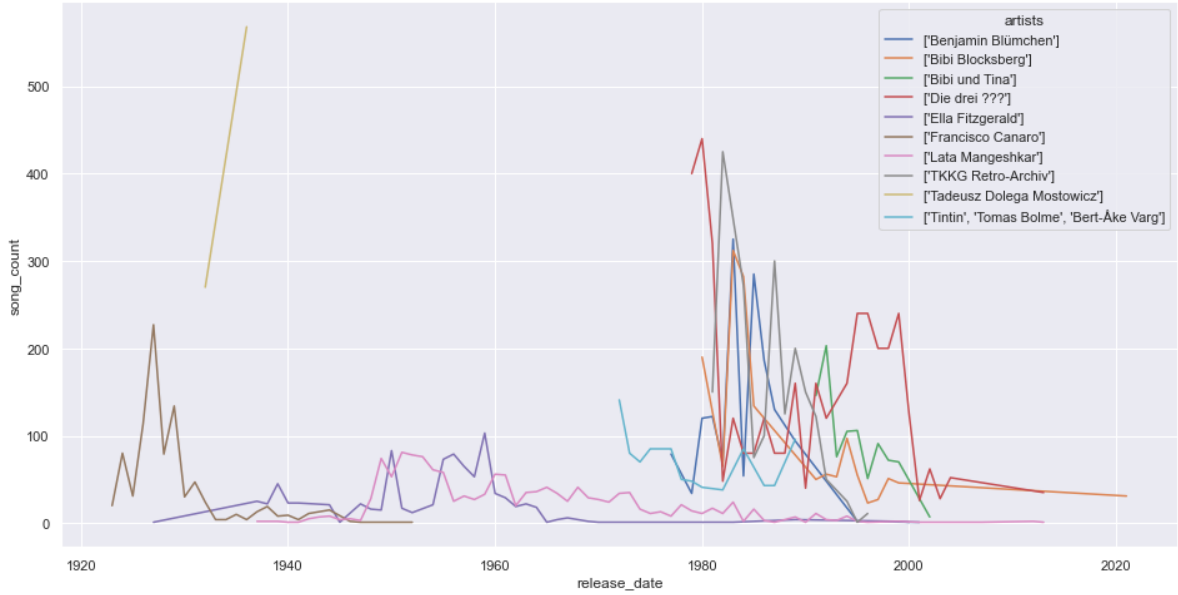
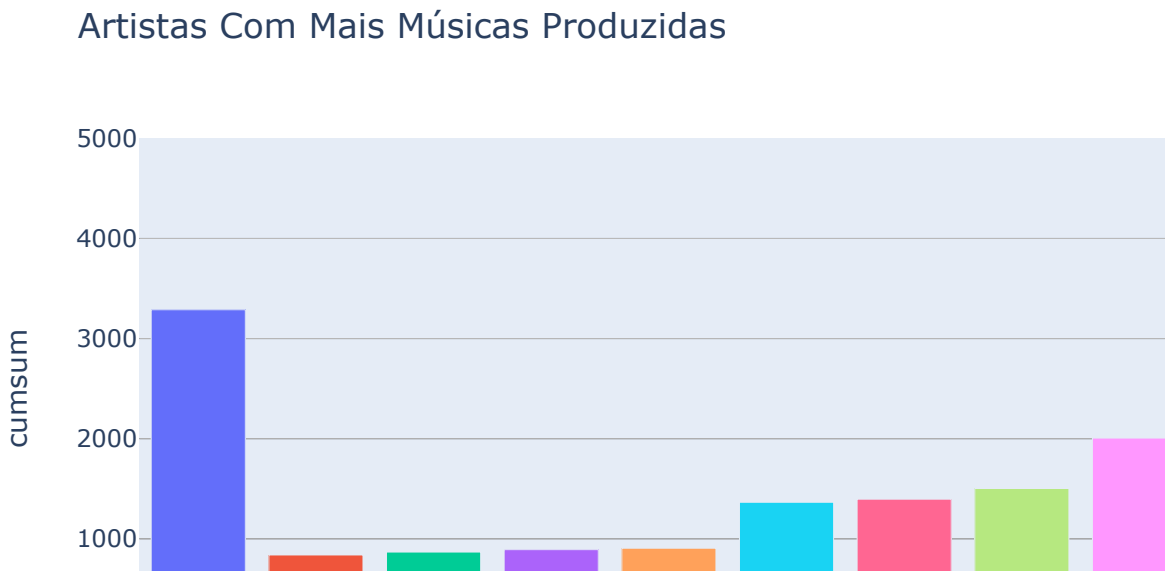
Análise: Demonstramos a partir do tratamento/desmebramento da coluna "artists" os 10 artistas com mais produções na nossa amostra.

observação:

Optamos por não desmembrar os dados da coluna "artists", pois esta ação requer muito processamento e se torna inviável. Porém, gostaríamos de deixar registrado a ideia da tratativa das informações do conjunto de dados. O conjunto de dados do Spotify possui mais de 580.000 registros contendo muitos artistas. Então pensamos em criar um gráfico animado de todo o período da coluna "release_date" (1900 a 2021) dos 10 artistas que mais produziram músicas. A ideia é mostrar a evolução/produção, ano a ano, de cada um desses artistas. Com isso, utilizamos uma técnica de acumular os valores num novo dataFrame e assim animar um gráfico de colunas. O ponto principal que nos motivou a registrar esse execício foi o desenvolvimento do gráfico animado como mostrado a seguir.

In [23]:

```
df = new_df.copy()

print(pd.Series(df["artists"].nunique()))

# Existem 114030 artistas em todo o conjunto de dados.
# Alguns deles produzem muitas canções, enquanto há alguns artistas com pouquíssimas.
# a) Vamos ver os 10 principais artistas com mais músicas no conjunto de dados.

# b) Podemos criar um novo dataframe que mostra a produção anual de canções para esses 10 a
# Primeiro criamos uma lista usando o índice retornado pela função value_counts

artist_list = df.artists.value_counts().index[:10]

df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
df['release_date'] = df['release_date'].dt.strftime('%Y').astype(int)

# Agora filtro  no dataframe usando esta lista e agrupando por ano:
df_artists = df[df.artists.isin(artist_list)][['artists','release_date','energy']].groupby(
df_artists.rename(columns={'energy':'song_count'}, inplace=True)

# Este dataframe contém o nome do artista, ano e quantas músicas o artista produziu naquele
# Agora vou criar um gráfico de linha comparando as produções desses artistas.

plt.figure(figsize=(16,8))
sbn.lineplot(x='release_date', y='song_count', hue='artists', data=df_artists)
plt.show()

# Não podemos realmente separar as linhas. Por ser um período tão longo (100 anos), os arti
# Verificarei quais artistas estão dominando em cada ano.
#
# Primeiro, vou criar um dataframe vazio que contém toda a linha do tempo (1900-2021) e os

df1 = pd.DataFrame(np.zeros((121,10)), columns=artist_list)
df1['release_date'] = np.arange(1900,2021)
#Converter as datas em int para realizar o merge com o mesmo tipo de dado
df1['release_date'] = df1['release_date'].astype(int)

# O dataframe inclui 121 linhas para 121 anos e 11 colunas (10 artistas e uma coluna de ano

df1 = df1.melt(id_vars='release_date',var_name='artists', value_name='song_count')

# A contagem de músicas é zero em todos os anos. Vou mesclar contagens de músicas do datafr
df_merge = pd.merge(df1, df_artists, on=['release_date','artists'], how='outer').sort_value

#Substituirei os valores NaN por 0 e eliminarei a coluna song_count_x.
df_merge.fillna(0, inplace=True)
df_merge.drop('song_count_x', axis=1, inplace=True)
df_merge.rename(columns={'song_count_y':'song_count'}, inplace=True)

# Também quero adicionar uma coluna que mostra a soma cumulativa das
# Músicas que cada artista produziu ao longo dos anos. Uma maneira de fazer isso é usar as
df_merge['cumsum'] = df_merge[['song_count','artists']].groupby('artists').cumsum()
df_merge.head(10)

fig = px.bar(df_merge,
             x='artists', y='cumsum',
             color='artists',
             animation_frame='release_date', animation_group='release_date',
             range_y=[0,5000],
```

```
            title='Artistas Com Mais Músicas Produzidas')
fig.show()
```

```
0    114029
dtype: int64
```